

Hotel Management System

Architecture Document

Next.js 15 + Supabase | Single Property | EN/NE

Version 1.0 — May 2026

1. Project Overview

This document describes the architecture of a hotel management system for a single property. The system enables guests to browse rooms, book stays with email OTP verification, and chat with reception. Staff manage operations through a role-based dashboard, and the super admin can update virtually all content (rooms, photos, menu, pricing, page copy, policies) without developer involvement.

The application is built with Next.js 15 (App Router) and Supabase. After deployment and handover, the client should not need to contact the developer for routine content or pricing changes.

1.1 Requirements Summary

Aspect	Decision
Scope	Single hotel property
User roles	Guest, Receptionist, Manager, Super Admin
Guest auth	No signup; email OTP at booking time, phone number also needed
Staff auth	Email OTP with 30-day remember-me on trusted devices
Payments	Online (Khalti + eSewa) and pay-at-hotel
Notifications	Email + in-app realtime notifications
Languages	English
Add-on features	Reviews, extra services, guest-reception chat
Cancellation	Refund handled by admin
Pricing	Admin/Manage defined per room types

2. Technology Stack

Each layer was chosen to minimize operational burden after handover and to support Nepal-specific requirements (local payments, Nepali language).

2.1 Frontend and Backend

- **Next.js 15 (App Router)** with TypeScript handles both the public site and API routes / server actions in one codebase.
- **Tailwind CSS** with the **shadcn/ui** component library for a consistent, themeable UI.
- **Zod** for input validation on every server action and webhook.

2.2 Database, Auth, Storage, Realtime

- **Supabase Postgres** stores all application data, with Row Level Security (RLS) policies on every table.
- **Supabase Auth** handles email OTP login, phone number is collected and stored alongside the email
- **Supabase Storage** stores room photos, food images, gallery, and any guest-uploaded documents.
- **Supabase Realtime** powers guest-to-reception chat and in-app notifications.

2.3 External Integrations

- **Khalti and eSewa** for online payments (Nepal-specific). Stripe can be added later for foreign cards.
- **SMS provider** (MSG91 or Twilio) for OTP delivery during booking and staff login.
- **Resend** for transactional email (booking confirmations, cancellations, review requests).

2.4 Deployment

- **Vercel** for the Next.js application (auto-deploy from main branch).
- **Supabase Pro tier** for production database, with daily backups.

3. User Roles and Permissions

Four roles exist in the system. Each role inherits permissions from the role below it.

Role	Permissions
Guest	• Browse rooms, services, food menu without login, menu is a public section of the site • Book rooms with email OTP verification • View own bookings via cookie session • Request extra services during stay (no in-app, food ordering, menu is browse only) • Submit reviews in google review after checkout • Chat with reception in real time
Receptionist	• Everything a Guest can do, plus: • Create walk-in bookings (with stub guest profiles) • Verify bookings via phone call • Check guests in and out • Respond to guest chats • View today's arrivals and departures
Manager	• Everything a Receptionist can do, plus: • CRUD on rooms, room types, food menu, services

	• set per room type price, manage cancellation and manually process refund • View reports (revenue, occupancy, top rooms) • Configure Google review
Super Admin	• Everything a Manager can do, plus: • Invite, disable, and assign roles to staff • Edit site content (homepage, pages, gallery, FAQs) • Edit branding (logo, colors, hotel name) • Edit email and notification templates • View audit logs • Configure app settings

4. Authentication Flow

4.1 Guest Flow (No Signup)

Guests can browse the entire public site without an account. The OTP gate only appears at the booking confirmation step.

- The guest fills the booking form with dates, room, name, email, and phone number. Both email and phone are required: email is used for OTP delivery, and the phone number is stored alongside the booking for staff contact and fraud reduction (so a bad actor cannot complete a booking with only a throwaway email).
- The system sends a 6-digit OTP via email (through Resend) to the entered email address. OTP is never delivered by SMS in this system.
- Guest enters the OTP; the system verifies and creates (or reuses) a stub profile keyed on email, with the phone number stored as a secondary identifier.
- Booking is saved with `verification_method = 'otp'`.
- A signed cookie valid for 30 days is set on the guest's device, allowing them to view My Bookings without re-verifying.

4.2 Staff-Initiated Booking (Phone Call)

If a guest cannot use email OTP (e.g., a walk-in or phone reservation without a usable email), staff can verify identity through a phone call.

- The receptionist takes details over the phone and creates a stub guest profile.
- Booking is saved with `verification_method = 'staff_call'` and `verified_by =` receptionist's user id.
- The same chat functionality is available to that guest later; the Google Review prompt is sent if a valid email is on file.

4.3 Staff Login

- Staff log in with email OTP, identical to guests.
- A 'Trust this device for 30 days' option issues a longer-lived signed cookie.
- On untrusted devices, OTP is required each session.

5. Database Schema

All tables live in a single Supabase Postgres database. Row Level Security is enabled on every table. Text fields (e.g., room name, description) are stored as plain text columns in English only. If a second language is required in the future, fields can either be duplicated or migrated to JSONB at that time — this is deferred until there is a clear need.

5.1 Core Tables

Table	Purpose
profiles	Extends auth.users. Stores full_name, phone, role (enum), avatar_url, locale preference.
room_types	Categories such as Deluxe, Suite, Family. Holds base_price (set manually by admin), max_guests, amenities, and English name/description.
rooms	Physical rooms. Holds room_number (unique), type_id, floor, status enum (available, occupied, maintenance, cleaning).
bookings	Reservations. Holds guest_id, room_id, check_in, check_out, guests_count, total_amount, status, payment_status, payment_method, verification_method.
payments	Each payment attempt. Holds booking_id, amount, method, provider (khalti, esewa, cash), transaction_id, status.
otp_verifications	Stores phone, code_hash, expires_at, attempts, purpose (booking, staff_login).

5.2 Add-on Feature Tables

Table	Purpose
food_items	Menu items shown on the public menu page (browse-only, no ordering). Name and description in English, price, category, image_url, is_available.
services	Spa, laundry, airport pickup/drop, etc. English name and description. Category enum.

service_requests	Booked services per stay, with scheduled_at and status.
google_reviews_cache	Periodically refreshed snapshot of Google Reviews pulled via the Places API (author name, rating 1-5, comment, timestamp, profile photo URL). Reviews themselves live on Google; this table only caches them for fast public-site rendering. No write path from guests.
conversations	One per guest. Tracks last_message_at and unread counters.
messages	Individual chat messages. Subscribed via Supabase Realtime.
notifications	Per-user in-app notifications. Realtime-backed.

5.3 CMS and Configuration Tables

These tables make the system self-managed. Everything user-facing that isn't transactional data lives here so the super admin can edit it.

Table	Purpose
site_settings	Singleton row: hotel name, logo, favicon, tagline, address, phone, email, social links, business hours, currency, timezone, tax_rate, service_charge_rate.
branding	Primary, secondary, and accent colors; font family. Loaded into CSS variables at runtime.
pages	Editable pages (home, about, contact, terms). Holds slug, meta_title, meta_description.
page_sections	Reorderable sections per page. section_type (hero, text, gallery, cta, faq), order, content (JSONB), is_visible.
gallery_images	Photo gallery shown on a public site. Caption, category, order.
amenities	Hotel-level amenities (wifi, parking, pool). English name, icon.
faqs	Frequently asked questions. English question and answer, category, order.
testimonials	Curated testimonials managed by admin (separate from the Google Reviews cache).
cancellation_policy	Tiered refund schedule: hours_before_checkin, refund_percentage.
email_templates	Editable transactional email copy with {{variable}} placeholders.
notification_templates	Same idea for in-app notifications.

app_config	Key/value store for feature flags, OTP expiry seconds, min stay, max guests per booking, etc.
audit_logs	Records super admin actions: who changed what, when. Visible only to Super Admin.

6. Pricing

Pricing is admin-controlled and intentionally simple. There is no automatic dynamic pricing engine: weekend uplifts, seasonal surcharges, holiday rates, and any other variations are decided and entered by the admin directly.

6.1 How Prices Are Set

1. Each room type has a single `base_price` stored on `room_types`, edited by the admin from the dashboard.
2. When the admin wants to change a price (for a season, holiday, or any other reason), they update the `base_price` directly. There is no rule engine, calendar override, or priority resolution.
3. At booking time the system multiplies `base_price` by the number of nights, then adds `tax_rate` and `service_charge_rate` from `site_settings` to produce the total.
4. Each booking snapshots the price that was active at the time of booking on the `bookings.total_amount` field, so later admin price changes do not affect existing reservations.

6.2 Admin UI for Pricing

- A single price field per room type on the room types edit page.
- Tax and service-charge percentages are edited once on the `site_settings` page.
- Every price edit is recorded in `audit_logs` with old and new values.

7. Cancellation Policy

Cancellation is governed by a tiered refund table. The admin edits these tiers from the dashboard. The system computes the recommended refund amount based on the matched tier, but it does not move money — the admin processes every refund manually outside the system.

7.1 Example Tier Configuration

Hours before check-in	Refund percentage	Fee deducted
More than 72 hours	100%	0%
24 to 72 hours	50%	50%

Less than 24 hours	0%	100%
--------------------	----	------

7.2 Cancellation Flow

- Guest (or staff) initiates cancellation on a confirmed booking.
- The system computes `hours_until_check_in` and looks up the matching tier.
- The system computes the recommended refund amount (`paid_amount × refund_percentage`) and records it on the booking as `refund_amount_due`. This is a calculation only — no money is moved by the system.
- The booking is marked as cancelled and the admin is notified. The admin then issues the refund manually outside the system (through Khalti/eSewa dashboards or in cash for pay-at-hotel bookings) and marks the booking as refunded with the actual `refunded_amount` and a reference note.
- Pay-at-hotel bookings: booking status is updated and staff is notified; nothing further is required from the system.
- Cancellation acknowledgment email fires to the guest stating the cancellation has been received and the refund will be processed manually by the hotel. A separate email fires once the admin marks the booking as refunded. All actions are logged in `audit_logs`.

8. Content Management Layer

The super admin can edit virtually every user-facing element from the dashboard without developer involvement. This is the 'zero-touch after handover' requirement.

8.1 What Can Be Edited

- Hotel name, logo, favicon, tagline, address, contact details, social links.
- Brand colors (primary, secondary, accent) and font family — applied via CSS variables at runtime.
- Homepage and other pages: section order, section visibility, all copy and images.
- Gallery: add, remove, reorder, caption photos.
- Amenities, FAQs, testimonials — full CRUD.
- Rooms, room types, room photos, room amenities.
- Food menu items with images and categories (displayed on the public menu page; not orderable).
- Services (spa, laundry, airport pickup).
- Room-type prices (single value per type) and cancellation tiers.
- Email and in-app notification copy with variable placeholders.
- App settings (OTP expiry, min stay, max guests, etc.).

8.2 What Cannot Be Edited from the Dashboard

To keep scope manageable, the following are not in scope for the v1 CMS:

- Page layout structure (the order and types of available sections are predefined).
- Adding new top-level pages beyond home, about, contact, terms (without a developer).
- Custom HTML or JavaScript injection.

If the client later needs a full page builder, that is a separate phase.

9. Folder Structure (not required)

Next.js App Router with route groups for role-based gating.

```
app/ (auth)/ login/ verify-otp/ (guest)/ rooms/ rooms/[id]/ booking/[id]/ my-bookings/
menu/ // public food menu, browse-only services/ chat/ (staff)/ dashboard/
bookings/ cancellations/ // mark refund issued, record reference rooms/ walk-in/
service-requests/ chat/ (manager)/ reports/ menu/ // edit menu items shown
on public page rooms-manage/ pricing/ // single price per room type (admin)/
staff/ content/ branding/ templates/ settings/ google-reviews/ // configure Place
ID, refresh cache audit/ api/ webhooks/ khalti/ esewa/ bookings/ payments/ cron/
refresh-google-reviews/ middleware.ts // role check via Supabase session lib/ supabase/ // client,
server, middleware payments/ // khalti, esewa adapters pricing/ // simple total = base_price * nights
+ tax + service google-reviews/ // Places API client and cache refresher validators/ // zod schemas
components/ ui/ // shadcn primitives feature/ // domain components types/ hooks/
```

10. Key User Flows

10.1 Booking and Payment

- Guest selects check-in and check-out dates on a room detail page.
- System multiplies the room type's admin-set price by the number of nights, adds tax and service charge, and shows the total.
- Guest chooses 'Pay online' or 'Pay at hotel'.
- Guest submits booking form (with email and phone); email OTP is sent and verified.
- Booking is created with status = 'pending'.
- If pay online, guest is redirected to Khalti or eSewa; on webhook confirmation booking becomes 'confirmed'.
- If pay at hotel, booking is immediately 'confirmed' but payment_status remains 'unpaid'.
- Email + in-app notification fires to guest and staff.

10.2 Check-in and Check-out

- Receptionist looks up the booking via guest name, phone, or booking code.
- Marks booking as 'checked_in'; room status flips to 'occupied'.
- On checkout, booking goes to 'checked_out'; room goes to 'cleaning'.

- Guest receives an email with a link to the hotel's Google Review page (along with the in-app thank-you notification).

10.3 Chat with Reception

- One conversation per guest, shared inbox for staff.
- Realtime subscription on the messages table; new messages appear instantly on both sides.
- Unread counters update via database triggers.

10.4 Reviews (Google Reviews integration)

- After a booking is marked 'checked_out', the system sends the guest an email containing the hotel's Google Review link (and optionally a short thank-you message editable in the email template).
- A scheduled job (daily) calls the Google Places API for the configured Place ID and refreshes the google_reviews_cache table. The public site renders reviews from this cache so page loads do not depend on Google's API uptime. Hiding or moderating a review is done on Google directly — the system does not have its own moderation UI.

11. Security Approach

11.1 Row Level Security

Every table has RLS enabled. Policies use Supabase's `auth.uid()` and a `is_staff()` helper function that reads from the `profiles.role` column.

- Guest policies typically read: 'guest can SELECT/INSERT/UPDATE their own rows'.
- Staff policies typically read: 'staff with role in (receptionist, manager, super_admin) can SELECT/INSERT/UPDATE'.
- CMS tables: only super_admin can write; everyone reads (since it powers the public site).

11.2 API and Webhook Security

- Payment webhooks (Khalti, eSewa) are verified with a shared secret signature on each request.
- All form input is validated server-side with Zod before hitting the database.
- Middleware blocks staff routes for non-staff users before page rendering.

11.3 Audit Trail

- All super admin write actions (price changes, content edits, staff changes) are recorded in `audit_logs` with actor, timestamp, entity, and old/new values.
- Booking and payment state transitions are also logged.

12. Build Phase Plan

Six phases, each independently shippable. Order is dependency-driven.

Phase	Timeline	Scope
1. Foundation	Week 1	• Next.js + Supabase project setup • Schema migration with English-only text fields • RLS policies, role enum • Base middleware • Seeded site_settings and branding
2. CMS Core	Week 2	• Super admin edits site_settings, branding, pages, sections • Amenities, gallery, FAQs CRUD • Single (English) input fields everywhere • Public homepage reads from CMS
3. Rooms + Booking	Weeks 3-4	• Room types, rooms CRUD (price is a single field per room type) • Public rooms listing and detail pages • Date picker with live availability • Simple total calculation (nights × admin-set price + tax + service) • Booking with email OTP (phone also captured) • My Bookings view
4. Payments	Weeks 4-5	• Khalti integration with webhook • eSewa integration with webhook • Cancellation flow with manual refund recording (no auto-refund API) • Cancellation policy enforcement • Pay-at-hotel flow
5. Staff Ops	Weeks 5-6	• Receptionist check-in/out • Walk-in bookings, phone-call verification • Today's arrivals and departures • Manager reports • Super admin staff management + audit log

6. Add-ons	Weeks 6-7	• Food menu CMS + public menu page (display only, no ordering) • Services CMS + requests • Google Reviews integration (Place ID config, daily cache refresh, public display) • Guest-reception chat • Editable email and notification templates
------------	-----------	---

After Phase 6: polish, accessibility audit, mobile responsiveness pass, security review, performance tuning, and deployment to Vercel + Supabase Pro tier.

13. Next Steps

Architecture is locked. From here the next options are:

- **Detailed ER diagram** showing all tables, columns, and relationships.
- **Complete route map** for every page and API endpoint, with role gates.

Begin Phase 1 by scaffolding the Next.js + Supabase project and writing the initial schema migration